

GSLC2-SNA
Implementasi CI/CD dan DevSecOps pada Flask API Docker

Mata Kuliah: Server and Network Administration

Kelompok 8 Anggota Kelompok
Maghfirli alif al ayubi - 2502022925
reyhan225– Lumban Tobing Reyhan Muhamad Rizki - 2802534241
Claudiuscpm – Claudius Cezar Panangian Manurung - 2802544646
SillyBuddy-004 – Muhammad Arifando Akbar - 2602179170

BAB I - PENDAHULUAN

Latar Belakang

Pada proses pengembangan aplikasi modern, automation menjadi salah satu hal yang penting karena dapat membantu developer dalam melakukan testing, deployment, dan monitoring aplikasi secara lebih cepat dan efisien.

Selain automation, faktor keamanan juga menjadi hal yang penting dalam development aplikasi. Oleh karena itu digunakan konsep DevSecOps, yaitu penggabungan antara development, security, dan operations dalam satu proses pipeline otomatis.

Pada tugas ini kami menggunakan project Flask API sederhana berbasis Docker yang sebelumnya sudah dibuat pada tugas Git dan Docker.

Tujuan

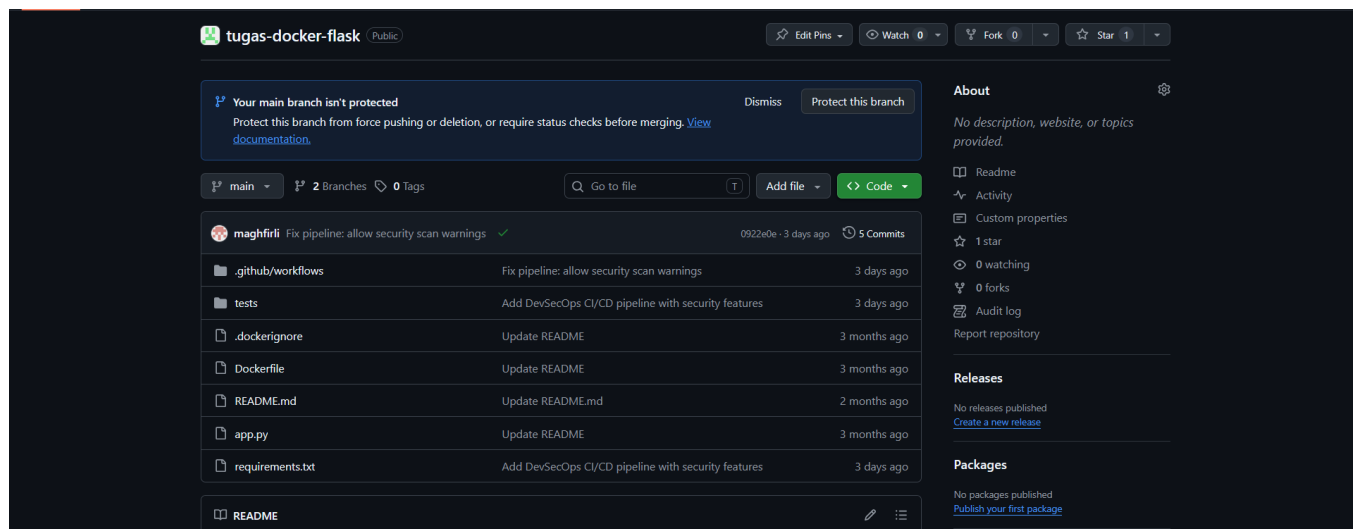
- Memahami implementasi CI/CD menggunakan GitHub Actions
- Memahami penerapan Docker dalam deployment aplikasi
- Memahami konsep dasar DevSecOps
- Menerapkan security scanning pada source code, dependency, dan Docker image

BAB II - DESKRIPSI PROJECT

Project yang digunakan merupakan sebuah backend API sederhana berbasis Python Flask. Aplikasi memiliki dua endpoint utama:

- `/` ☐ menampilkan response utama aplikasi
- `/health` ☐ digunakan untuk mengecek status aplikasi

Aplikasi dijalankan menggunakan Docker container sehingga environment aplikasi dapat berjalan dengan konsisten di berbagai device atau operating system.

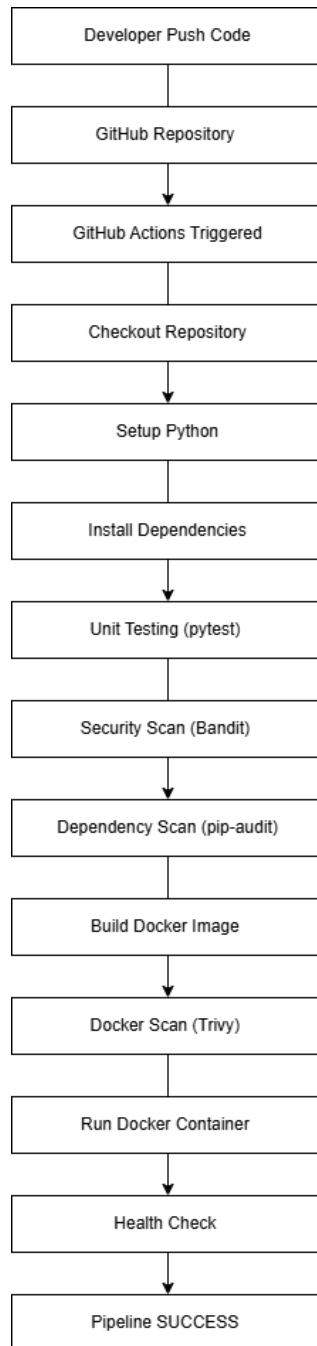


Gambar 1. Repository GitHub Project Flask API Docker

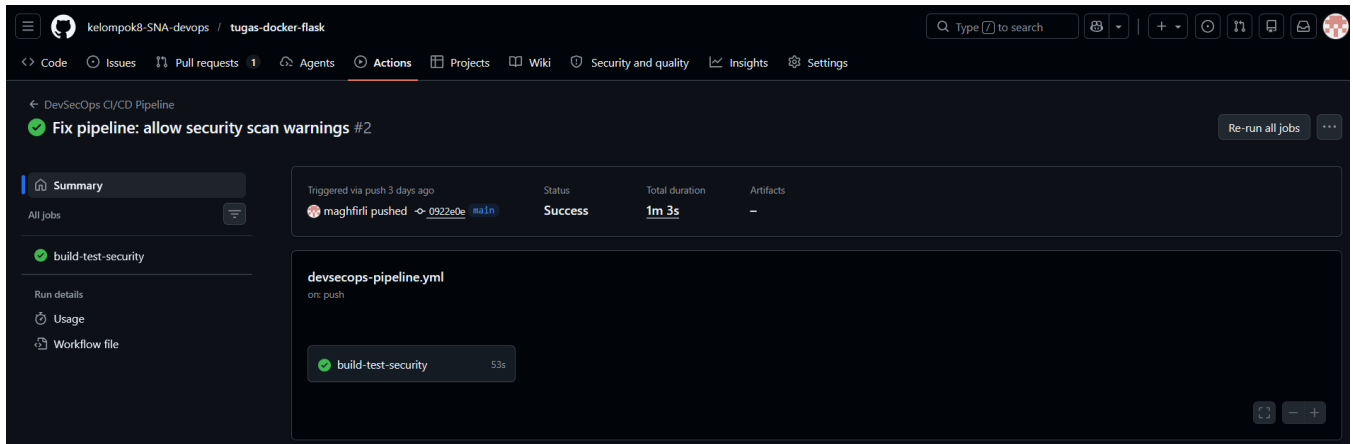
BAB III - IMPLEMENTASI CI/CD

CI/CD merupakan metode automation yang digunakan agar proses integration, testing, dan deployment aplikasi dapat berjalan secara otomatis.

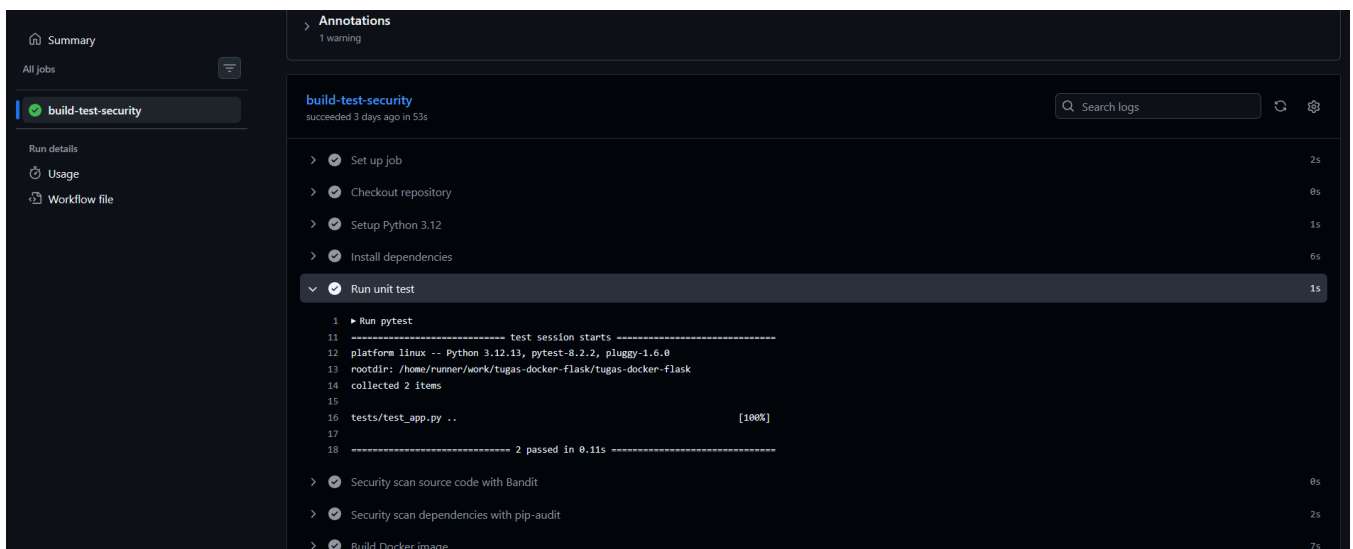
Pipeline dibuat menggunakan GitHub Actions dan berjalan otomatis setiap ada push ke branch main.



Gambar 2. Diagram Pipeline



Gambar 3. Hasil Running Pipeline GitHub Actions

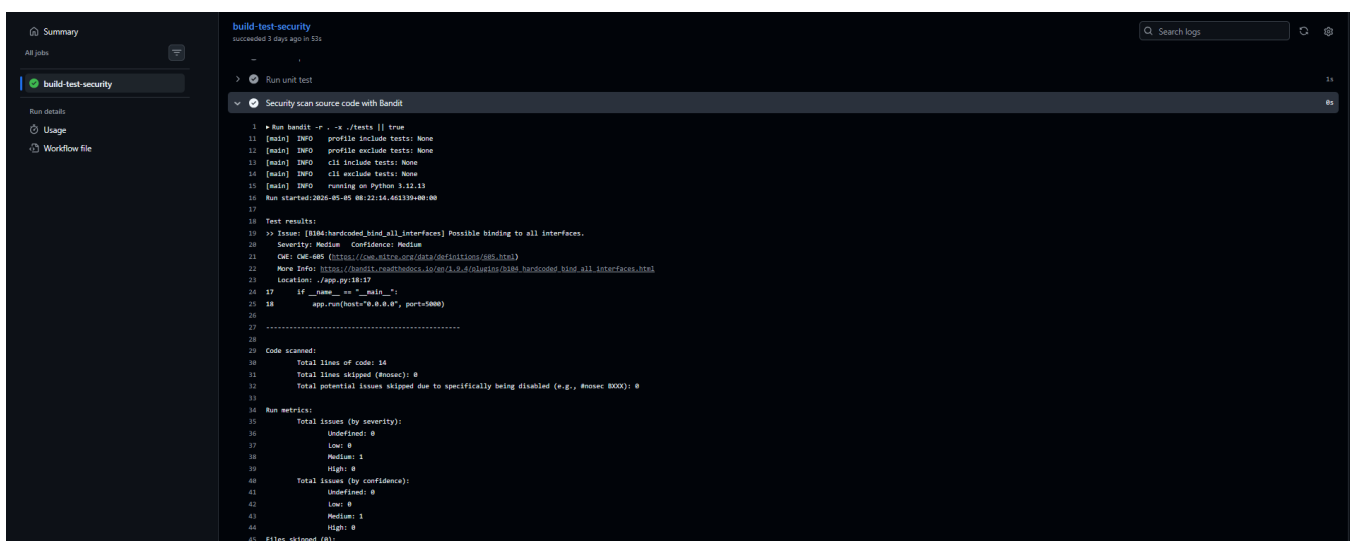


Gambar 4. Hasil Unit Testing menggunakan pytest

BAB IV - IMPLEMENTASI DEVSECOPS

Bandit

Bandit digunakan untuk melakukan static code analysis pada source code Python.



Gambar 5. Hasil Security Scan menggunakan Bandit

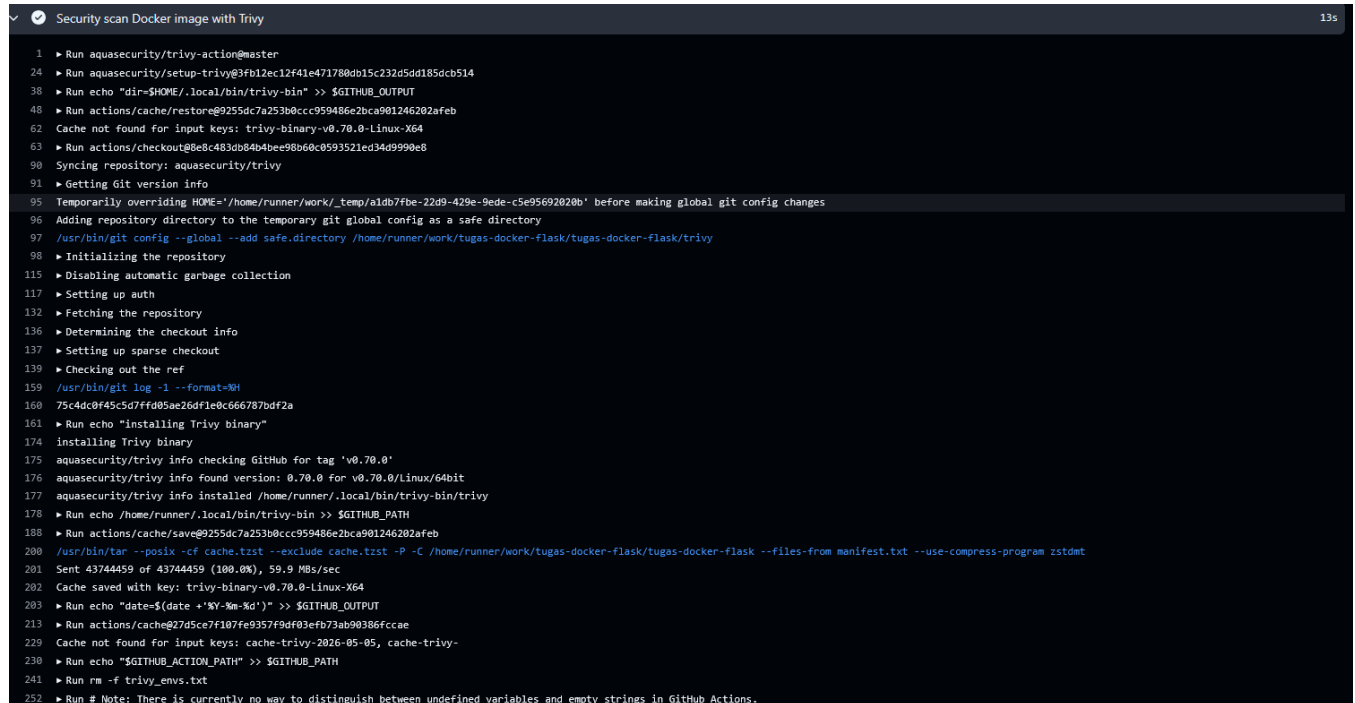
pip-audit

pip-audit digunakan untuk mengecek vulnerability pada dependency Python.

[MASUKKAN GAMBAR PIP-AUDIT DI SINI]
Gambar 6. Hasil Dependency Scan menggunakan pip-audit

Trivy

Trivy digunakan untuk melakukan scanning terhadap Docker image.



```
1 ▶ Run aquasecurity/trivy-action@master
24 ▶ Run aquasecurity/setup-trivy@93fb12ec12f41e471780db15c232d5dd185dcb514
38 ▶ Run echo "dir=$HOME/.local/bin/trivy-bin" >> $GITHUB_OUTPUT
48 ▶ Run actions/cache/restore@9255dc7a253b0cc959486e2bca901246202afeb
62 Cache not found for input keys: trivy-binary-v0.70.0-Linux-X64
63 ▶ Run actions/checkout@8e8c483db4b4bee98b60c0593521ed34d9990e8
90 Syncing repository: aquasecurity/trivy
91 ▶ Getting Git version info
95 Temporarily overriding HOME='/home/runner/work/_temp/a1db7fbc-22d9-429e-9ede-c5e95692020b' before making global git config changes
96 Adding repository directory to the temporary git global config as a safe directory
97 /usr/bin/git config --global --add safe.directory /home/runner/work/tugas-docker-flask/tugas-docker-flask/trivy
98 ▶ Initializing the repository
115 ▶ Disabling automatic garbage collection
117 ▶ Setting up auth
132 ▶ Fetching the repository
136 ▶ Determining the checkout info
137 ▶ Setting up sparse checkout
139 ▶ Checking out the ref
159 /usr/bin/git log -1 --format=%H
160 75c4dc0f45c5d7fd05ae26df1e0c666787bdf2a
161 ▶ Run echo "installing Trivy binary"
174 installing Trivy binary
175 aquasecurity/trivy info checking GitHub for tag 'v0.70.0'
176 aquasecurity/trivy info found version: 0.70.0 for v0.70.0/Linux/64bit
177 aquasecurity/trivy info installed /home/runner/.local/bin/trivy-bin/trivy
178 ▶ Run echo /home/runner/.local/bin/trivy-bin >> $GITHUB_PATH
188 ▶ Run actions/cache/save@9255dc7a253b0cc959486e2bca901246202afeb
200 /usr/bin/tar --posix -cf cache.tzst --exclude cache.tzst -P -C /home/runner/work/tugas-docker-flask/tugas-docker-flask --files-from manifest.txt --use-compress-program zstdmt
201 Sent 43744459 of 43744459 (100.0%), 59.9 MBs/sec
202 Cache saved with key: trivy-binary-v0.70.0-Linux-X64
203 ▶ Run echo "date=$(date +%Y-%m-%d)" >> $GITHUB_OUTPUT
213 ▶ Run actions/cache@27d5ce7f107fe9357f9df03efb73ab90386fcae
229 Cache not found for input keys: cache-trivy-2026-05-05, cache-trivy-
230 ▶ Run echo "$GITHUB_ACTION_PATH" >> $GITHUB_PATH
241 ▶ Run rm -f trivy_envs.txt
252 ▶ Run # Note: There is currently no way to distinguish between undefined variables and empty strings in GitHub Actions.
```

Gambar 7. Hasil Docker Image Scan menggunakan Trivy

BAB V - HASIL IMPLEMENTASI

Pipeline berhasil dijalankan menggunakan GitHub Actions dengan status SUCCESS.

Semua proses berjalan dengan baik:

- unit testing berhasil
- security scanning berhasil dijalankan
- Docker image berhasil dibuat
- container berhasil dijalankan
- endpoint /health berhasil diakses



```
1 ▶ Run curl http://localhost:5000/health
11 % Total % Received % Xferd Average Speed Time Time Time Current
12 Dload Upload Total Spent Left Speed
13
14 0 0 0 0 0 0 0 0 --:--:-- --:--:-- --:--:-- 0
15 100 16 100 16 0 0 7909 0 --:--:-- --:--:-- --:--:-- 8000
16 (*status:"ok")
```

Gambar 8. Hasil Health Check Endpoint

BAB VI - KESIMPULAN

Berdasarkan hasil implementasi yang dilakukan, dapat disimpulkan bahwa penggunaan CI/CD pipeline menggunakan GitHub Actions dapat membantu proses development menjadi lebih otomatis dan terstruktur.

Selain itu, dengan menambahkan fitur security seperti Bandit, pip-audit, dan Trivy, aplikasi dapat diperiksa dari sisi keamanan sejak awal proses development.

Walaupun project ini masih sederhana, konsep yang digunakan sudah sesuai dengan implementasi dasar DevSecOps pada dunia industri.

Link Repository GitHub

<https://github.com/kelompok8-SNA-devops/tugas-docker-flask.git>

